

Epic 1 : Model Selection and System Architecture

Story 1 : Research and select the Model: Gemini 1.5 Pro model from Google Generative AI for legal document generation.

Story 2 : Define architecture: Frontend: Streamlit for user input and document download. o Backend: FastAPI ([main.py](#), [routes.py](#)) to handle API requests. AI Integration: Gemini 1.5 API used inside [ai_core/gemini_generator.py](#).

Story 3 : Set up development environment with .env file containing the Gemini API key.

Epic 2 : Core Functionalities Development

Story 1 : Implement core generation features: Legal document creation o Editable text preview Format & export as .txt, .docx, and .pdf

Story 2 : Build FastAPI backend: Endpoint /generate to process POST requests o Accepts document_type, parties, terms, dates Returns generated text from Gemini

Epic 3 : [main.py](#) & API Logic

Story 1 : Create and configure [main.py](#): Initialize FastAPI app Route / for home Include router from [routes.py](#)

Story 2 : Handle document generation logic in [routes.py](#): Define Pydantic model DocumentRequest ,Call GeminiDocumentGenerator.generate_document(...)

Epic 4 : Frontend with Streamlit

Story 1 : Design user-friendly Streamlit interface: Input fields for all required data Responsive layout with title and logo

Story 2: Dynamic document generation preview: Show styled editable output Option to export as .txt, .docx, or .pdf

Epic 5 : Deployment

Story 1 : Prepare deployment: Install production-ready dependencies

Set up Procfile, requirements.txt, or Dockerfile if needed

Story 2 : Deploy application:

Options: Render, Railway, Deta, Streamlit Community Cloud (frontend), [Fly.io](#) or VPS for backend

Epic 6 : Conclusion

Story 1: Finalize and test: o Ensure end-to-end functionality

Story 2 :Test Gemini integration under different input combinations

Understand the Project Requirements

1. LegalEase must generate structured legal documents (e.g., contracts, NDAs, agreements) based on user inputs such as:

Document type

Parties involved

Terms and conditions

Effective dates

Explore [Gemini Model Documentation](#) o Visit Google Generative AI Documentation to evaluate available Gemini models. Review capabilities of gemini-1.5-pro, especially: Long context window (up to 1 million tokens) Structured prompting with multi-part content Fast generation speed and legal-domain compatibility

Evaluate Model Performance Benchmarks indicate that gemini-1.5-pro is capable of handling complex formatting, reasoning, and structured text generation, making it suitable for legal content generation.

Select the Optimal Model Selected: gemini-1.5-pro

Best suited for drafting formal documents with clear sections and legal jargon. Capable of understanding and generating clauses based on user inputs dynamically.

1. Architectural Diagram

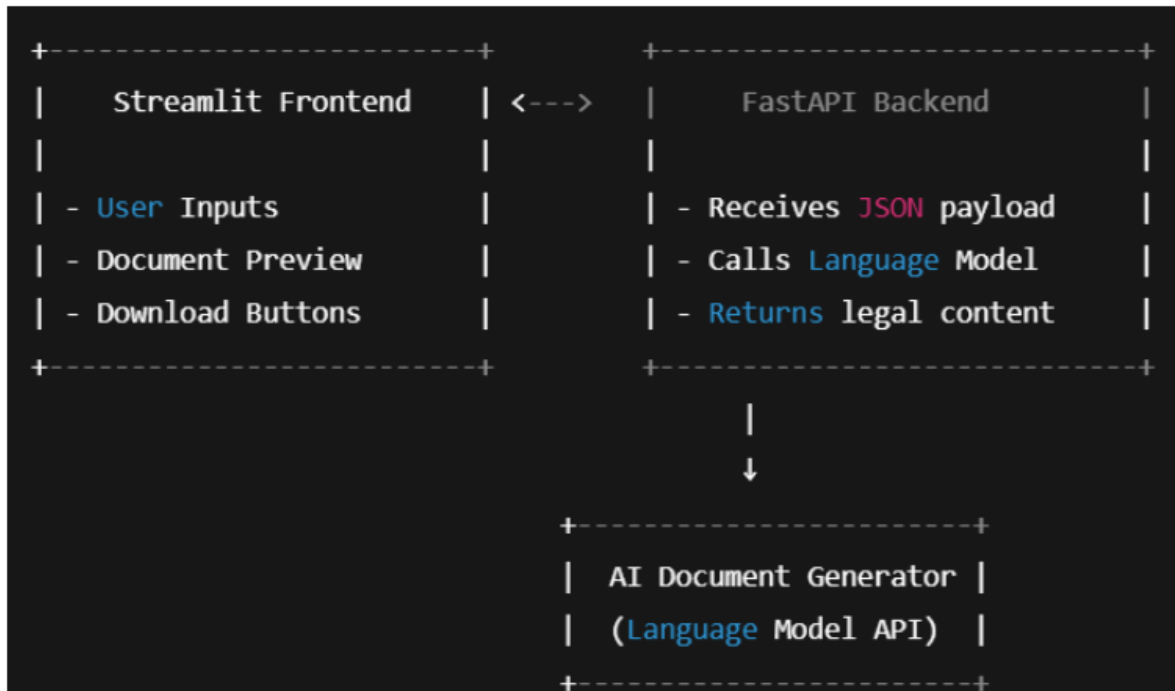
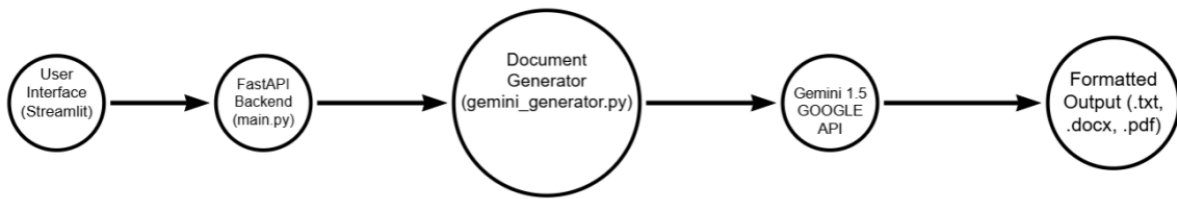
Components:

Frontend: Streamlit for user interaction (document input, preview, editing, downloads).

Backend: FastAPI app (routes input to Gemini model and returns generated content).

AI Core: Gemini integration for content generation Formatting.

Modules: DOCX, PDF, TXT generators.



2. Frontend Functionality

Users provide inputs via:

TextInput for document type and dates

TextArea for parties and terms

Buttons: "Generate Document"

"Edit Document"

"Download as TXT / DOCX / PDF"

Styled HTML preview of the generated legal document

3. Backend Responsibilities

Expose POST /generate endpoint (via FastAPI)

Accept JSON input (document_type, parties, terms, dates)

Route to GeminiDocumentGenerator

Return structured document text

4. AI Integration Points

Gemini Document Generator forms a structured prompt and sends it to Gemini via generate_content

Receive generated text and returns to FastAPI

Frontend displays and allows editing or exporting

1. Install Python and Pip Ensure Python 3.10+ is installed along with pip for dependency management.

Create a Virtual Environment

Install FastAPI and Uvicorn (Backend Server) pip install fastapi uvicorn

Install Streamlit (Frontend UI)

4. pip install streamlit

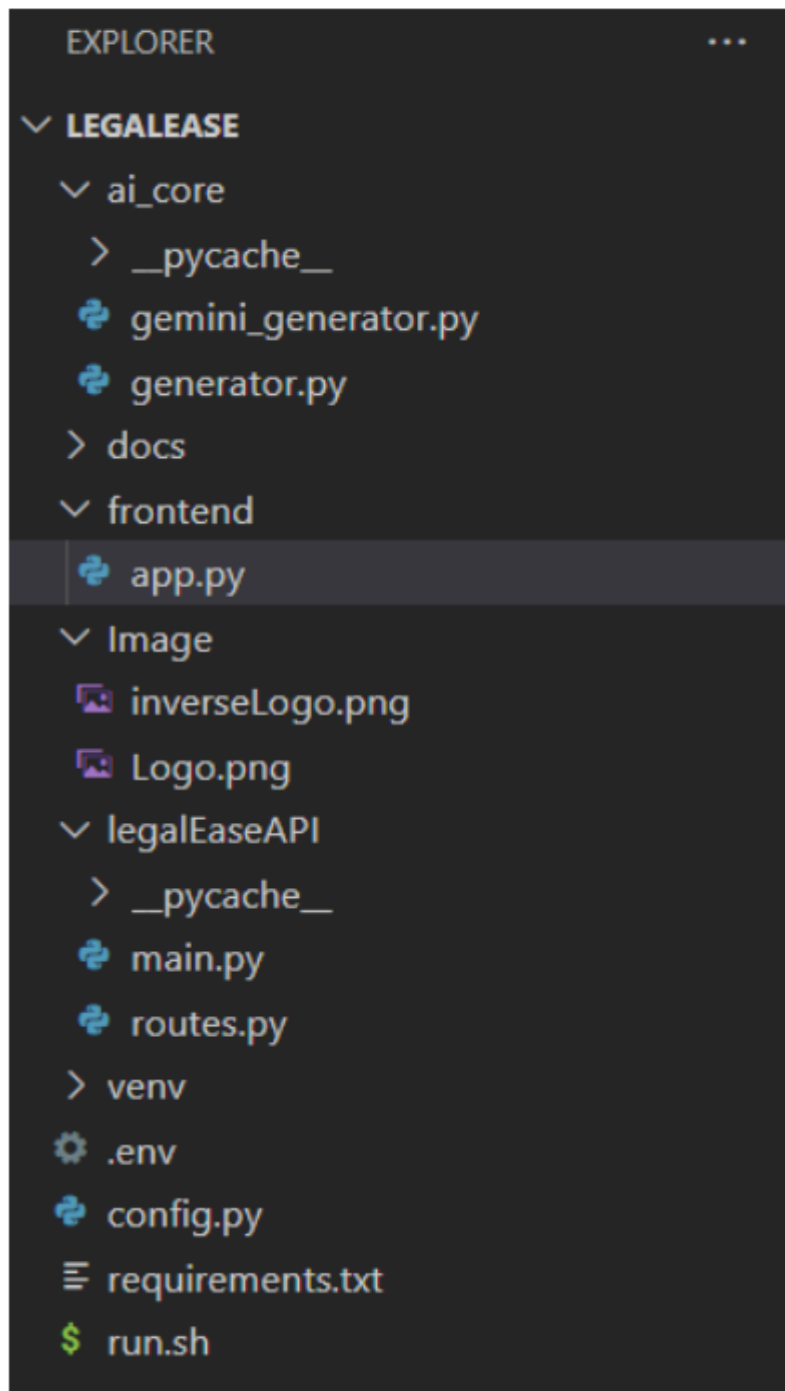
5. Install Google Generative AI SDK pip install google-generativeai

6. Install Additional Libraries pip install python-docx fpdf Pillow requests python-dotenv

7. Set Up Application Structure

1. Legal Document Generation

Functionality: Generate legal documents (e.g., contracts, agreements) based on user inputs (document type, parties, terms, and dates).



1. Legal Document Generation

Functionality: Generate legal documents (e.g., contracts, agreements) based on user inputs (document type, parties, terms, and dates).

```
# gemini_generator.py
def generate_document(self, document_type, parties, terms, dates):
    prompt = (
        f"Generate a comprehensive legal document titled '{document_type}'\n"
        f"Involved parties: {parties}\n"
        f"Effective Date: {dates}\n"
        f"Terms and conditions: {terms}\n"
        f"Ensure formal legal structure with multiple sections and legal clauses."
    )
    response = self.model.generate_content(prompt)
    return response.text
```

```
# routes.py
@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    response = gemini_generator.generate_document(
        request.document_type,
        request.parties,
        request.terms,
        request.dates
    )
    return {"document": response}
```

Document Output Formatting

Functionality: Convert raw AI-generated text into structured .docx, .pdf, .txt files with legal formatting, logos, tables, and footers. · format_docx(text, doc_type) ·

format_pdf(text, doc_type) · format_html_preview(text) These use: · docx for Microsoft Word formatting (includes logo, Times New Roman font, terms as table, footer). · fpdf for clean, branded PDF documents with headers/footers. · HTML Preview with Streamlit styling.

User Interaction & Recommendations

Functionality: Accept user preferences through the Streamlit interface and reflect them in the AI-generated legal content.

```
# app.py - Streamlit UI
document_type = st.text_input("Document Type")
parties = st.text_area("Parties Involved")
terms = st.text_area("Terms & Conditions")
dates = st.text_input("Effective Date")

# Send to backend
response = requests.post("http://localhost:8000/generate", json={
    "document_type": document_type,
    "parties": parties,
    "terms": terms,
    "dates": dates
})
```

4. Editable Document & Download

Functionality: Provides editable text area, allows download in .txt, .docx, .pdf formats.

```
# app.py
if st.session_state.get("show_edit"):
    edited_text = st.text_area("Edit Document Below:", generated_text, height=300)
    st.session_state.generated_text = edited_text

# Downloads
st.download_button("📄 Download as .TXT", data=generated_text, ...)
st.download_button("📄 Download as .DOCX", data=format_docx(...), ...)
st.download_button("📄 Download as .PDF", data=format_pdf(...), ...)
```

1. Define Routes in FastAPI (Instead of Flask)

```
# main.py
app.include_router(router)

# routes.py
@router.post("/generate") # Route for generating legal document
```

2. Process User Input

Handled via Pydantic & Streamlit Forms:

```
# routes.py
class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str
```

```
# app.py
# Streamlit collects input → FastAPI handles it
response = requests.post("http://localhost:8000/generate", json={...})
```

Integrate Groq (Gemini) API Calls

```
# gemini_generator.py
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

self.model = genai.GenerativeModel(model_name)
response = self.model.generate_content(prompt)
```

This step involves initializing the FastAPI application, setting up the base endpoint for health-checks, and including modular API routes.

Steps:

Initialize the FastAPI app

Define a root (/) endpoint to verify the service is running

Include routes from the [routes.py](#) module

```
from fastapi import FastAPI
from legalEaseAPI.routes import router

app = FastAPI(title="LegalEase - AI Legal Document Generator")

# Include API routes
app.include_router(router)

# Root endpoint
@app.get("/")
def home():
    return {"message": "Welcome to LegalEase AI Legal Document Generator API"}

# Run FastAPI if executed directly
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

This task handles user inputs from the frontend and sends them to the Gemini model for document generation. Steps:

- Define a Pydantic DocumentRequest model to validate incoming data.
- Initialize the GeminiDocumentGenerator from ai_core.gemini_generator.

- Create a POST endpoint (/generate) to accept user input and return generated legal content.

```
from fastapi import APIRouter
from pydantic import BaseModel
from ai_core.gemini_generator import GeminiDocumentGenerator

router = APIRouter()
gemini_generator = GeminiDocumentGenerator()

class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str

@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    response = gemini_generator.generate_document(
        request.document_type,
        request.parties,
        request.terms,
        request.dates
    )
    return {"document": response}
```

Step 1: Streamlit Page Configuration and Layout Setup

- Configured Streamlit app with:

```
st.set_page_config(page_title="LegalEase", layout="centered")
```

Added a three-column layout using st.columns to center-align the company logo

```
col1, col2, col3 = st.columns([1, 2, 1])
with col2:
    st.image(WEB_LOGO_PATH, use_container_width=True)
```

Step 2: Header and Title

- Displayed a subtle center-aligned title under the logo:

```
st.markdown(
    "<h2 style='text-align: center;'>AI Legal Document Generator</h2>",
    unsafe_allow_html=True
)
```

Step 3: User Input Interface

- Created intuitive inputs for:
 - o document_type: type of legal document
 - o parties: involved stakeholders
 - o terms: semicolon-separated clauses
 - o dates: effective date

Step 1: Generate Document with Backend AI

- On clicking "Generate Document", a POST request is sent to the FastAPI backend:

```
response = requests.post("http://localhost:8000/generate", json={...})
```

AI-generated content is sanitized and displayed dynamically:

```
generated_text = sanitize_text(response.json()["document"])
```

Step 2: HTML Preview Rendering

- The output is previewed in an elegant dark-themed scrollable card with semantic HTML:

```
styled_html = format_html_preview(generated_text)
st.markdown(f"<div style='...'>{styled_html}</div>", unsafe_allow_html=True)
```

Step 3: Editable Document Preview

- The user can click "Edit Document" and modify the generated content using a text area:

```
edited_text = st.text_area("Edit Document Below:", generated_text, height=300)
```

Step 4: Multi-Format Download Options

- Users can download the document as:
 - o .txt (Plain Text)
 - o .docx (Formatted Word Document with logo, headings, and table)
 - o .pdf (Branded PDF with logo and footer)

```
st.download_button("📄 Download as .TXT", data=generated_text, file_name=f"{document_type.replace(' ', '_').lower()}.txt",  
st.download_button("📄 Download as .DOCX", data=format_docx(generated_text, document_type), file_name=f"{document_type.replace(' ', '_').lower()}.docx",  
st.download_button("📄 Download as .PDF", data=format_pdf(generated_text, document_type), file_name=f"{document_type.replace(' ', '_').lower()}.pdf",
```

Core Functions & Utilities `sanitize_text(text)` • Removes special characters and typographic quotes to ensure clean formatting. `format_docx(text, doc_type)`

- Uses python-docx to:
 - o Embed logo
 - o Add titles, paragraph formatting
 - o Auto-generate a Terms table from semicolon-separated input
- `format_pdf(text, doc_type)`
 - o Utilizes FPDF with custom header/footer:
 - o Center-aligned logo
 - o Bold headings for sections
 - o Bullet-style terms
- `format_html_preview(text)`
 - o Converts output to stylized HTML blocks for inline display.

Step 1: Set Up a Virtual Environment

To ensure environment isolation and consistent dependency management, create and activate a virtual environment:

```
PS F:\VS CODE\python\LegalEase> venv\scripts\activate  
(venv) PS F:\VS CODE\python\LegalEase> |
```

Step 2: Install Dependencies

Ensure that the backend (FastAPI) and frontend (Streamlit) dependencies are installed from a requirements.txt:

```
pip install -r requirements.txt
```

```

requirements.txt
1  aiohappyeyeballs==2.6.1
2  aiohttp==3.11.16
3  aiosignal==1.3.2
4  altair==5.5.0
5  annotated-types==0.7.0
6  anyio==4.9.0
7  asgiref==3.8.1
8  attrs==25.3.0
9  backoff==2.2.1
10 bcrypt==4.3.0
11 blinker==1.9.0
12 build==1.2.2.post1
13 cachetools==5.5.2
14 certifi==2025.1.31
15 charset-normalizer==3.4.1
16 chroma-hnswlib==0.7.6
17 chromadb==1.0.0
18 click==8.1.8
19 colorama==0.4.6
20 coloredlogs==15.0.1
21 dataclasses-json==0.6.7

```

Step 3: Configure Environment Variables

Create a .env file to securely store your **Gemini API key**

Step 1: Launch the FastAPI Backend

Start the backend server using **Uvicorn**:

```

(venv) PS F:\VS CODE\python\LegalEase> uvicorn legalEaseAPI.main:app --reload
INFO: Will watch for changes in these directories: ['F:\VS CODE\python\LegalEase']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)INFO:
      Started reloader process [15168] using WatchFiles
INFO: Started server process [17360]
INFO: Waiting for application startup.
INFO: Application startup complete.

```

Step 2: Launch the Streamlit Frontend

Run the [app.py](#) file to start the **LegalEase** UI:

```

(venv) PS F:\VS CODE\python\LegalEase> streamlit run frontend/app.py

You can now view your Streamlit app in your browser.

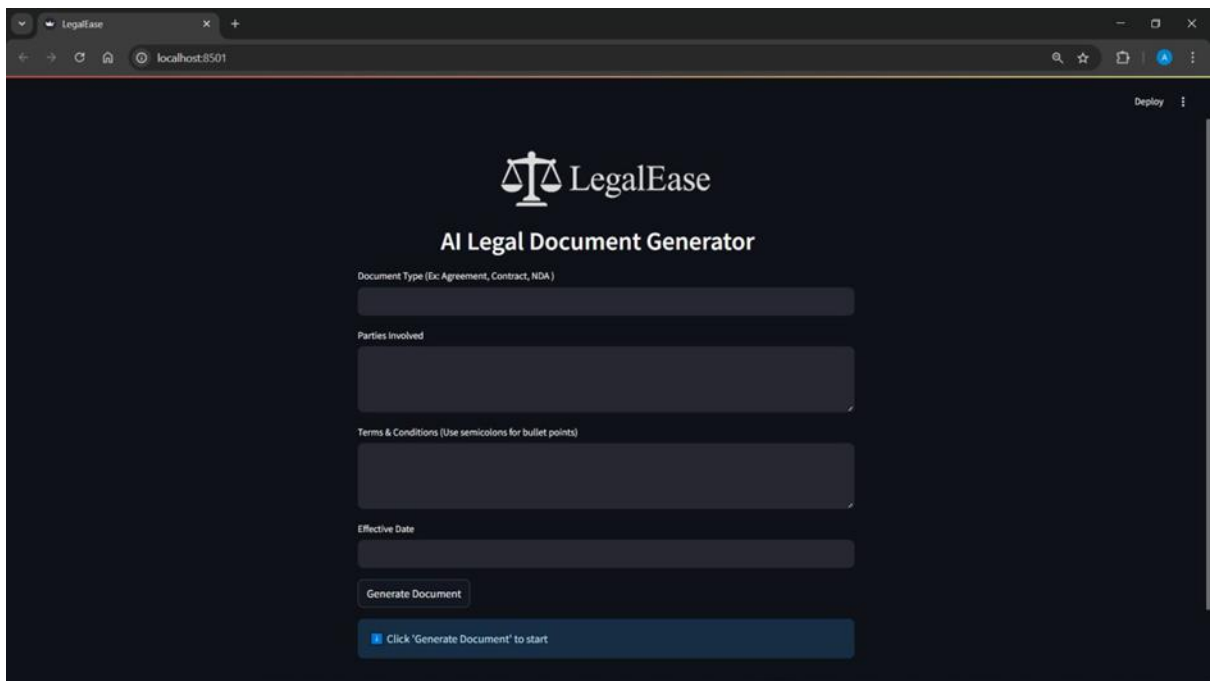
Local URL: http://localhost:8501
Network URL: http://192.168.1.3:8501

```

Step 3: Test All Functionalities

- Input document type, parties, terms, and date.
- Press **Generate Document** to get AI-generated legal content from Gemini.
- Use built-in options to:
 - o Preview in dark mode HTML
 - o Edit the text
 - o Download in .txt, .docx, and .pdf formats
- Check for:
 - o Proper layout with logo and footer
 - o Clean formatting of bullet points
 - o Correct embedding of terms in the document

Exploring LegalEase



The screenshot shows a web browser window with the URL 'localhost:8501'. The application is titled 'LegalEase' and 'AI Legal Document Generator'. It features a dark theme with a white logo of scales of justice. The form includes four input fields: 'Document Type (Ex: Agreement, Contract, NDA)', 'Parties Involved', 'Terms & Conditions (Use semicolons for bullet points)', and 'Effective Date'. Below these fields is a 'Generate Document' button and a blue button that says 'Click \'Generate Document\' to start'.

1. Document Type

Purpose: Type of legal document to generate. Examples:

- Agreement
- Contract
- NDA (Non-Disclosure Agreement)
- Lease Agreement
- Employment Offer Letter Example input: Freelance Work Contract

2. Parties Involved

Purpose: Names and roles of the individuals or entities involved in the document. Examples:

- John Doe (Freelancer), ABC Corp (Client)
- Alice Smith (Tenant), XYZ Realty (Landlord)

Example input: Jane Doe (Service Provider), TechNova Inc. (Client)

3. Terms & Conditions (Use semicolons for bullet points)

Purpose: Specific clauses or rules that define the agreement. Separate each bullet point using a semicolon.

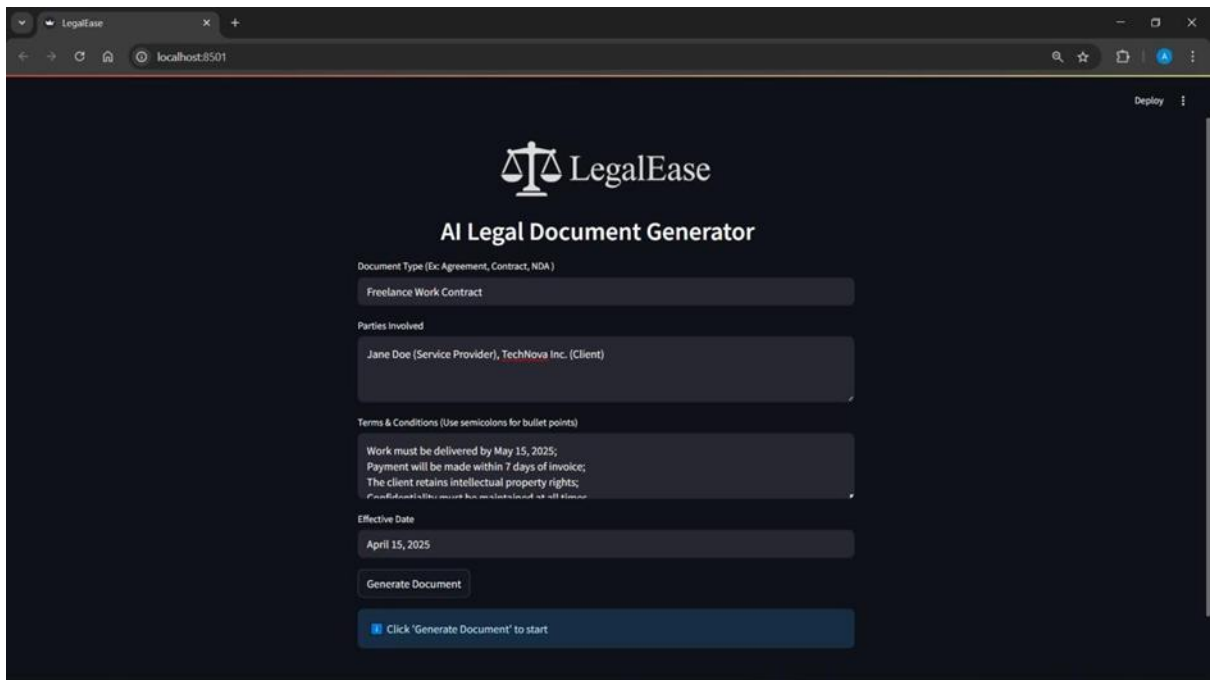
Examples:

- Payment to be made within 30 days of invoice;
- The provider agrees to deliver work by the agreed deadline;
- Confidentiality must be maintained at all times;
- Either party may terminate with 15 days notice

4. Effective Date

Purpose: The date when the agreement becomes legally valid. Examples:

- April 10, 2025
- 10/04/2025
- April 15, 2025



The screenshot shows a web browser window with the URL 'localhost:8501'. The application is titled 'LegalEase AI Legal Document Generator'. It features a dark theme with white text. The interface includes several input fields and a 'Generate Document' button. The input fields are labeled: 'Document Type (Ex: Agreement, Contract, NDA)' with the value 'Freelance Work Contract'; 'Parties Involved' with the value 'Jane Doe (Service Provider), TechNova Inc. (Client)'; 'Terms & Conditions (Use semicolons for bullet points)' with the value 'Work must be delivered by May 15, 2025; Payment will be made within 7 days of invoice; The client retains intellectual property rights; Confidentiality must be maintained at all times.'; and 'Effective Date' with the value 'April 15, 2025'. Below the input fields is a 'Generate Document' button. At the bottom, there is a blue button with a white icon and the text 'Click \'Generate Document\' to start'.

Document Generated

Effective Date

April 15, 2025

Generate Document

✔

 Document Generated Successfully!

Freelance Work Contract

Agreement made this 15th day of April, 2025

Between:

Jane Doe (hereinafter referred to as the "Service Provider"), residing at [Jane Doe's Address], [Jane Doe's City, State, Zip Code]

And:

TechNova Inc. (hereinafter referred to as the "Client"), a corporation organized and existing under

✎

 Click to Edit Document

📄

 Download as .TXT

📄

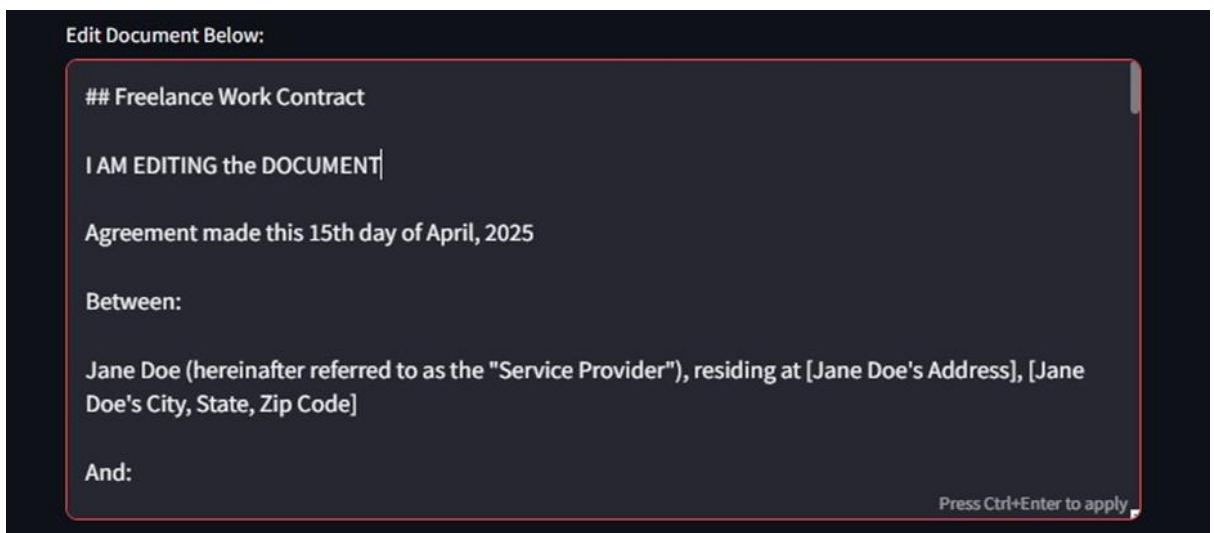
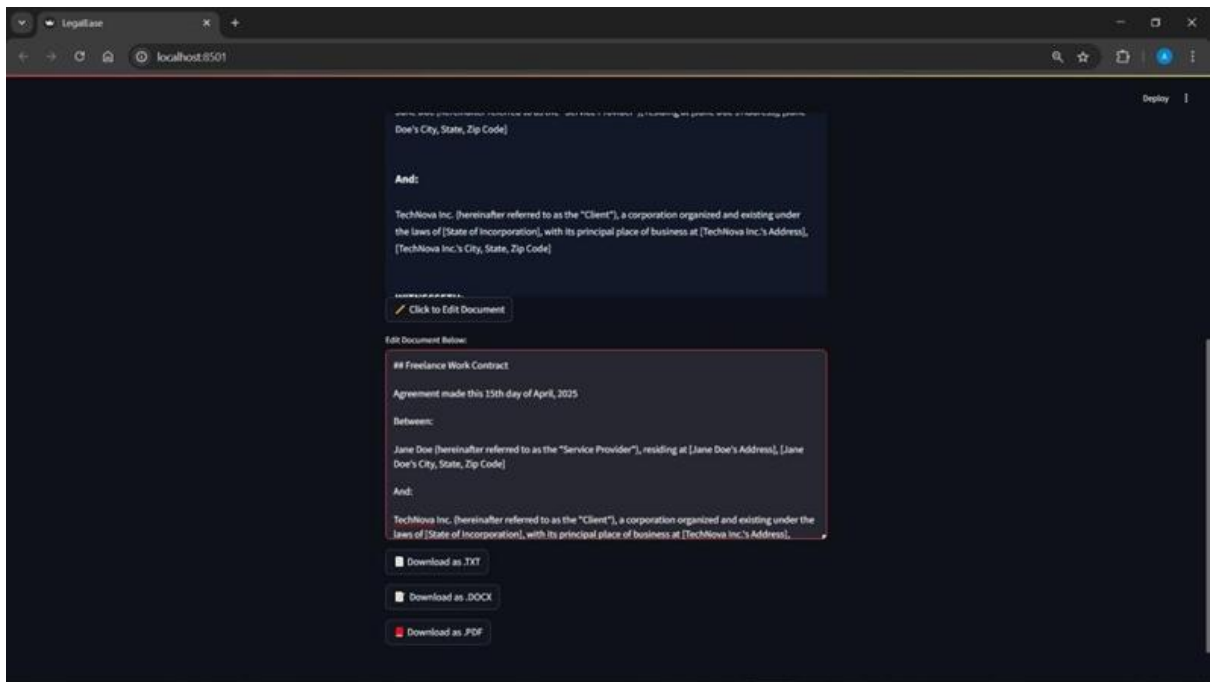
 Download as .DOCX

📄

 Download as .PDF

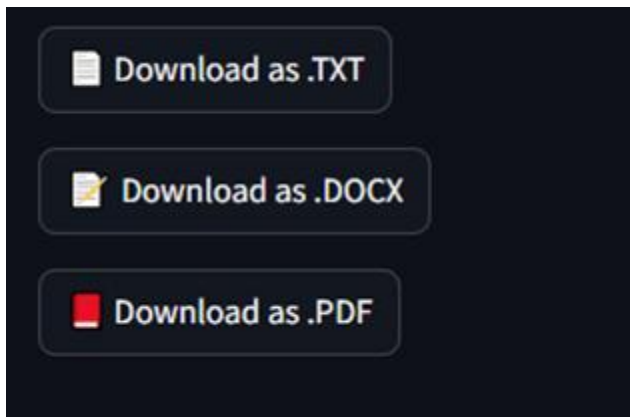
Editing the Document

If the user wants to edit the document, they can do so by clicking on the “ Click to Edit Document” Botton. A new window opens up where the document can be edited.



Downloading/ Saving the Generated Document

We can Download and save the documents in any of the following formats.



.txt

```
freelance_work_contract - Notepad
File Edit Format View Help
### Freelance Work Contract

Agreement made this 15th day of April, 2025

Between:

Jane Doe (hereinafter referred to as the "Service Provider"), residing at [Jane Doe's Address], [Jane Doe's City, State, Zip Code]

And:

TechNova Inc. (hereinafter referred to as the "Client"), a corporation organized and existing under the laws of [State of Incorporation], with its principal place of business at [TechNova Inc.'s Address], [TechNova Inc.'s City, State, Zip Code]

WITNESSETH:

WHEREAS, the Client desires to engage the Service Provider to perform certain services as described herein; and

WHEREAS, the Service Provider is willing to perform such services for the Client on the terms and conditions set forth in this Agreement;

NOW, THEREFORE, in consideration of the mutual covenants and promises contained herein, the parties agree as follows:

1. Services:

The Service Provider agrees to provide the following services to the Client (the "Services"): [Clearly and comprehensively describe the services to be rendered, including specific tasks, deliverables, and any required milestones.]

2. Term and Termination:

This Agreement shall commence on the Effective Date and shall terminate upon the completion of the Services as defined in Section 1, or upon the occurrence of any of the following events:

a) Mutual written agreement of the parties;
b) Breach of this Agreement by either party, provided that the non-breaching party provides written notice of the breach and allows the breaching party [Number] days to cure the breach;
c) Upon [Number] days written notice by either party for any reason.

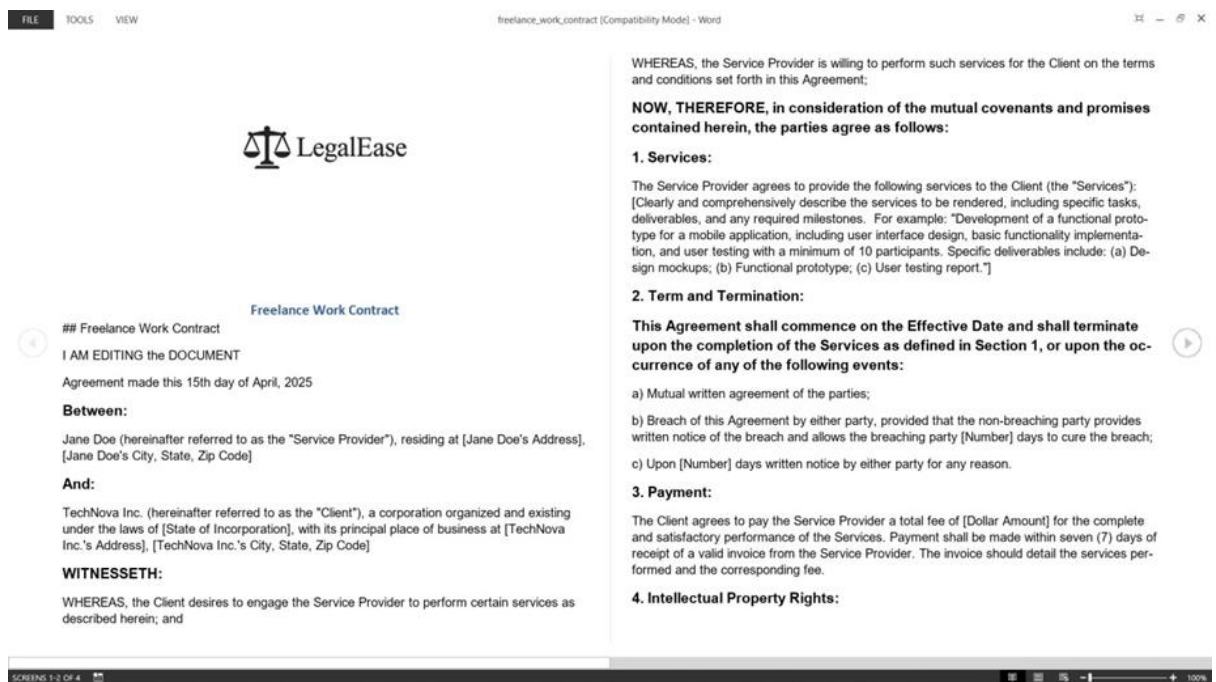
3. Payment:

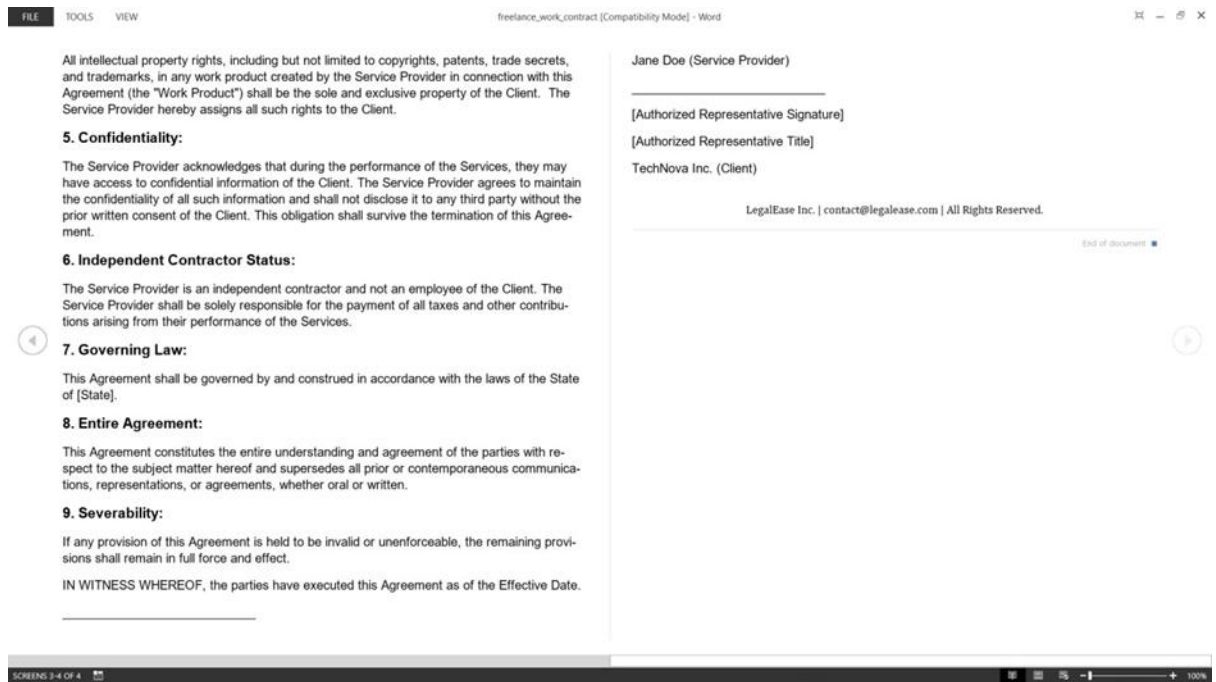
The Client agrees to pay the Service Provider a total fee of [Dollar Amount] for the complete and satisfactory performance of the Services. Payment shall be made within seven (7) days of receipt of a valid invoice from the Service Provider.

4. Intellectual Property Rights:
```

.docx

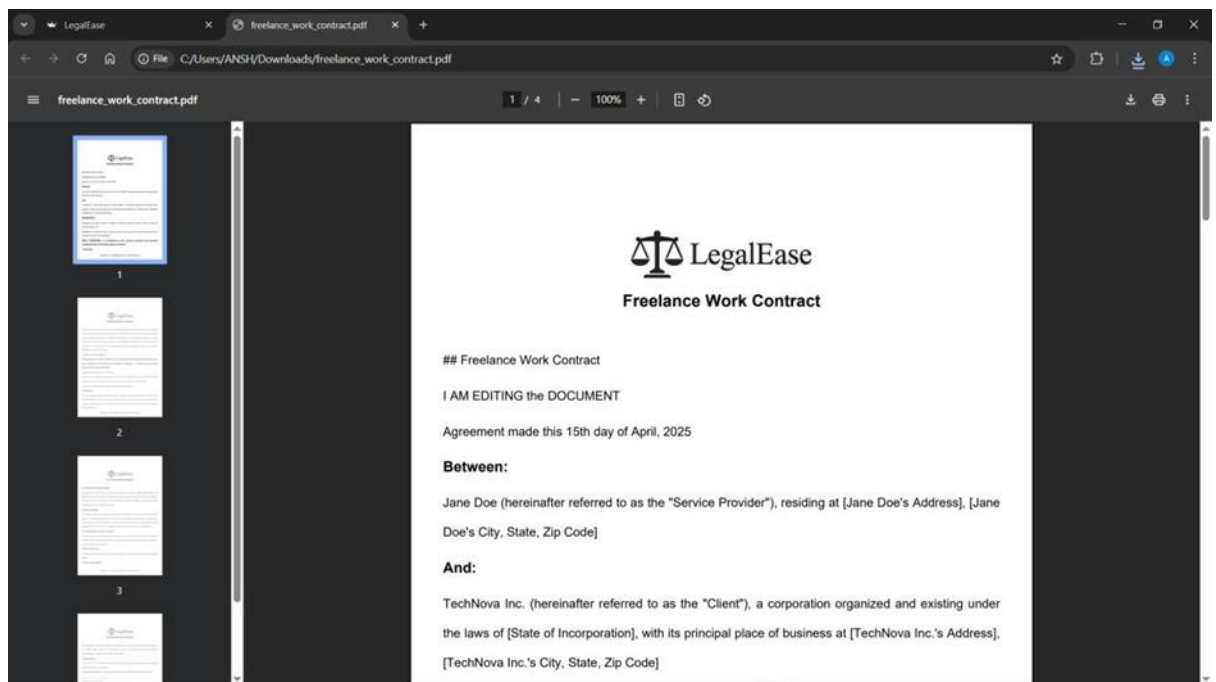
The document provided inculcates the use of the logo (front page) as well as the footer in the last page

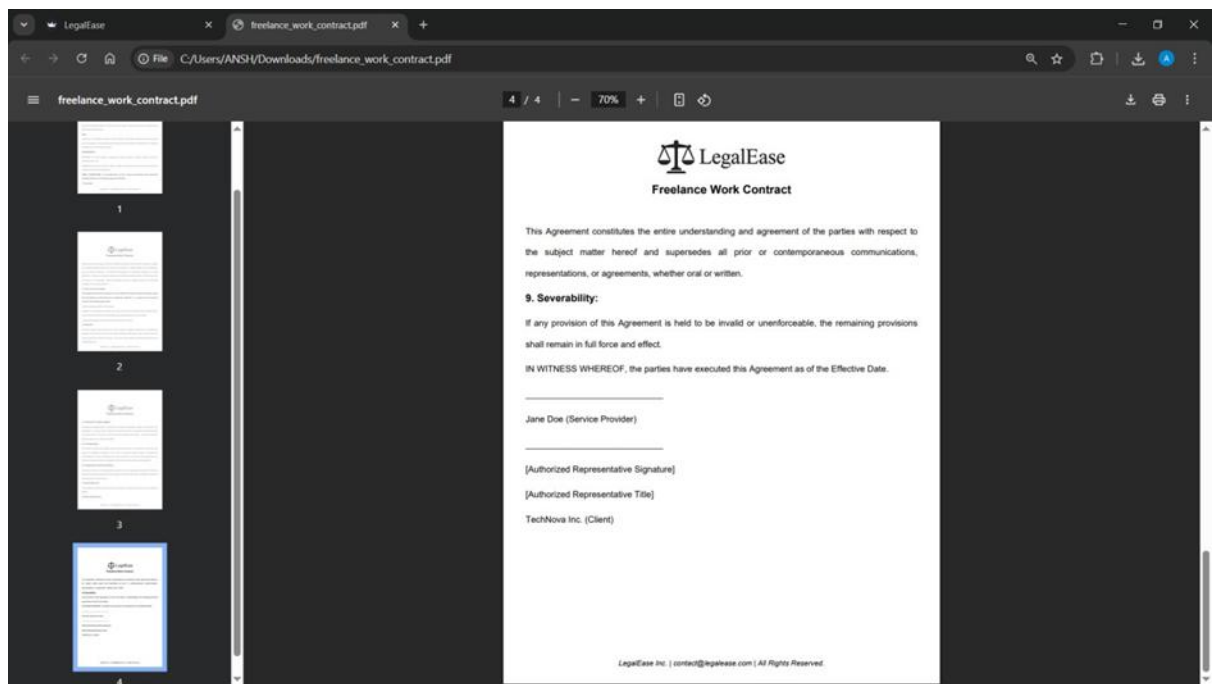




.pdf

This format provided the user with a well formatted and ready to use legal document with Logo and Footer on all the pages.





LegalEase represents a transformative step in bridging the accessibility gap in legal services through the power of artificial intelligence. This project demonstrates how emerging technologies can simplify complex legal language, providing users—regardless of legal background—with a clear, concise understanding of legal documents. By integrating advanced natural language processing (NLP) models with user-centric design, LegalEase empowers individuals to navigate legal content with confidence, thereby promoting inclusivity and informed decision-making.

The platform's ability to analyze and simplify legal text, highlight key clauses, and provide plain-language summaries enhances comprehension and reduces dependency on costly legal consultations for basic understanding. This democratization of legal knowledge is critical in a world where legal literacy remains low, especially among marginalized populations. Furthermore, LegalEase supports multilingual capabilities, ensuring broader reach and utility across different regions and demographics.

From a technical standpoint, LegalEase showcases efficient integration of backend AI models, including document parsing, summarization algorithms, and user-friendly web interfaces. The project's architecture is designed to ensure scalability, data privacy, and real-time responsiveness—key elements for any modern SaaS solution. Through iterative development, testing, and feedback, the system has been refined for optimal performance and user satisfaction.

In conclusion, LegalEase is more than a technological tool—it is a catalyst for legal empowerment. By fostering transparency, accessibility, and understanding, it upholds the principle that legal information should be a public good, not a privileged resource. Future enhancements could include deeper contract analysis, integration with legal databases, and personalized recommendations. With continued innovation and support, LegalEase has the potential to revolutionize how people interact with the law, making justice more approachable for all.